

Your project companion

A simple reference for Omkar Joshi. Learn what to type, what it does, and when you need it. Examples use ByteLabs on Windows. Prepared 4 October 2026.

Five words to remember

Repository: your project plus its saved Git history.

Branch: a named line of work, such as main or demo.

Commit: a recorded checkpoint on the current branch.

Local: on your computer.

Remote: a repository elsewhere, such as GitHub. origin is its usual nickname.

The everyday journey

Edit files -> git add -> git commit -> git push
Computer GitHub

Saving a file in your editor does not make a commit. A commit is local until you push it. Pushing does not merge a different branch into main.

GitHub is not your live website

Git tracks changes. GitHub stores and reviews them. Render builds and publishes your website. A successful push does not prove deployment succeeded: check Render, then open the live site.

Read this guide by situation

Page 2: check your project and save work.

Page 3: download, upload, and understand branches.

Page 4: recommended daily workflow with a pull request.

Page 5: your demo/main case and working directly on main.

Page 6: common errors and careful recovery.

Page 7: setup, website commands, and quick reference.

Before typing: run commands inside the project folder, one at a time. If a command reports an error, stop and understand it before continuing. Names such as fix-mobile are examples you can replace.

Check first. Save second.

```
cd /d E:\Porjects_Update\Bytelabs
```

Meaning: Opens your project folder in Windows Command Prompt.

Use it when: You opened a new CMD window. In PowerShell use: Set-Location E:\Porjects_Update\Bytelabs

```
git status
```

Meaning: Shows the current branch, staged changes, unstaged changes, and new files.

Use it when: Before switching, committing, pulling, or pushing.

```
git diff
```

Meaning: Shows edits in tracked files that are not staged. It does not show new untracked file contents.

Use it when: You want to review your changes before selecting them.

```
git add src/App.css src/ByteLabs.jsx
```

Meaning: Selects these files for the next commit.

Use it when: You want to include only specific files.

```
git add .
```

Meaning: Selects additions, edits, and deletions under the current folder.

Use it when: All those changes belong in your next commit. Review first; do not include passwords or .env secrets.

```
git diff --staged
```

Meaning: Shows the tracked content selected for your next commit.

Use it when: You want one final check before committing.

```
git commit -m "Fix mobile heading and email links"
```

Meaning: Records your staged changes on your current local branch.

Use it when: You have finished a meaningful piece of work. It does not upload anything.

Good messages: say what changed: "Add Google verification file" is clearer than "update". Unstaged edits are not included in the commit.

Connect your computer to GitHub

```
git fetch --prune origin
```

Meaning: Downloads current remote branch information and removes stale tracking references. It does not merge files or delete your local branches.

Use it when: You want fresh information before checking ahead/behind.

```
git branch -vv
```

Meaning: Lists local branches, their latest commits, and tracking information. The star marks your current branch.

Use it when: You are unsure where you saved your work.

ahead 1: one local commit is not in the tracked remote branch.

behind 2: the tracked remote branch has two commits you lack.

Important: tracking information can be old until you fetch.

```
git switch main
```

Meaning: Switches to local main. It does not download GitHub updates.

Use it when: You want to work on main or update it. Commit or stash unfinished work first.

```
git pull --no-rebase origin main
```

Meaning: Fetches GitHub main and integrates it into your CURRENT branch using merge behavior.

Use it when: You are on main and want its latest updates. Check git status first.

Why --no-rebase? It explicitly chooses merge, preserving existing commits. If only GitHub has new commits, Git may simply move forward; if both sides have new commits, it may create a merge commit. Conflicts are still possible. Plain git pull follows your Git configuration.

```
git push -u origin fix-mobile
```

Meaning: Uploads fix-mobile and sets its tracking branch. It can create that branch on GitHub.

Use it when: The first push of a new local branch. Later, git push is enough when tracking is configured.

```
git push origin main
```

Meaning: Uploads local main commits to GitHub main.

Use it when: You intend to update main and direct pushes are allowed. It does not upload commits that exist only on demo.

Recommended daily workflow

Use a separate branch for each change. This keeps main stable and lets you review work before adding it to the main project.

1. Start from an updated main

```
git status
git switch main
git pull --no-rebase origin main
git switch -c fix-mobile
```

Start with a clean working tree. The last command creates fix-mobile from your current main and switches to it. Use a fresh name for each task.

2. Edit and check the website

```
npm run dev
# Edit your files and check desktop and mobile layouts.
```

Open the local address printed by the development server. Stop it with Ctrl+C when needed. Check the changed behavior, not just how it looks.

3. Record and upload the change

```
git status
git diff
git add .
git diff --staged
git commit -m "Fix mobile heading and email links"
git push -u origin fix-mobile
```

4. Create a pull request on GitHub

Open your repository, then Pull requests > New pull request. Set **base: main** and **compare: fix-mobile**. Review the diff, create the PR, and merge when ready. A PR is a request to combine branches; creating one does not merge it.

5. Bring the merge back to your computer

```
git switch main
git pull --no-rebase origin main
git branch -d fix-mobile
```

Delete the local branch only after its work is integrated. If -d refuses, do not force deletion: inspect first. A squash merge can cause this even when the changes are present. Check Render deployment if this updates the live site.

What happened to your demo branch?

Based on the output you shared, main pointed to the earlier PR merge. Your local demo pointed to a newer commit called "Errors Fixed". These were separate branch histories.

```
main: earlier changes merged through PR #2
demo: earlier changes + "Errors Fixed"
```

Deleting demo on GitHub did not delete demo on your computer. Committing on local demo saved your fixes there. Pushing main could not upload those demo-only commits. "Up to date" described main, not every branch. "4 hours ago" was a commit timestamp, not an error.

Option A: use another PR

```
git switch demo
git push -u origin demo
```

This recreates the GitHub demo branch if needed. Create a PR with base main and compare demo, then merge it. Update local main afterwards. This works with rules that require pull requests.

Option B: merge locally and push main

```
git status
git switch main
git pull --no-rebase origin main
git merge demo
git push origin main
```

Use this only if direct main pushes are allowed and the working tree is clean. git merge demo adds demo history into your current branch. Stop if it reports conflicts. After success, git branch -d demo can remove the local branch.

Future work directly on main

```
git switch main
git pull --no-rebase origin main
# Edit and test.
git add .
git commit -m "Describe the change"
git push origin main
```

Choose one approach, not both. A branch-and-PR workflow is useful even for a solo project because you can review changes before merging.

When something goes wrong

"Rejected (fetch first)" or "non-fast-forward"

GitHub has history your local branch lacks. Do not force-push to bypass it. For main, check git status, ensure you are on main, then pull --no-rebase origin main. Resolve any conflict, review and test, then push again.

"Nothing to commit, working tree clean"

No uncommitted changes exist in this checkout. It does not mean every branch was uploaded. Check git branch -vv and git log -1 --oneline. Confirm the file was saved and you are in the right project.

Merge conflicts

Both histories changed overlapping content. Run git status to identify files. Open them and resolve the sections between conflict markers: <<<<<<, =====, and >>>>>>. Keep the correct result and remove the markers. Then stage the resolved files, run checks, and git commit. If unsure, stop and ask for help.

```
git merge --abort
```

Use during an unfinished merge to cancel it. Start merges with a clean working tree so cancelling is more predictable. Do not run this after a completed merge expecting it to undo history.

Unstage a file without deleting your edits

```
git restore --staged src/App.css
```

Removes the file from the next commit selection. Your edited file remains on disk. This example assumes the repository already has a commit.

Temporarily put unfinished work aside

```
git stash push -u -m "Unfinished mobile work"
git stash list
git stash apply
# After checking the restored work: git stash drop
```

Stash saves tracked edits and, with -u, untracked files. It does not upload them or include ignored files. apply restores the latest stash but keeps it as a backup; it may cause conflicts. Only drop after checking.

Commands to understand before using

git restore FILE discards unstaged edits in that tracked file.

git reset --hard, **git clean -fd**, **git branch -D**, **git push --force** can discard work or overwrite history. They are not routine fixes for a rejected push.

Keep this page nearby

Getting an existing project onto a new computer

```
git clone https://github.com/omkarjoshi21/Bytelabs.git
cd Bytelabs
npm install
```

clone creates a new project folder with Git history. Do not clone into your existing project folder. npm install reads package.json and installs dependencies; run scripts only from a project you trust.

First-time Git identity

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Replace both values. They identify your commits, not your GitHub login. Use your GitHub-provided no-reply email if you want privacy. --global applies to your user account; omit it to configure only this repository.

Useful inspection commands

```
git remote -v
git log --oneline -5
git log --oneline origin/main..demo
git diff main...demo
```

In order: show remote addresses; show five recent commits; show demo commits absent from your last fetched origin/main; show demo changes since its common ancestor with main. Fetch first for a fresh remote comparison.

Website commands for this ByteLabs project

```
npm run dev      # local development server
npm run lint     # code checks
npm run test     # interaction tests
npm run build    # create production files
npm run preview  # locally preview an existing build
```

The # comments above explain each line; you can type just the command. These scripts come from this project's package.json and may differ in another project. preview is not deployment. Keep node_modules, generated dist files, and secrets out of commits according to your .gitignore.

Before you leave your project

Check the branch. Review the changes. Run relevant checks. Commit with a clear message. Push the correct branch. If needed, merge the PR. Check hosting deployment and test the live page.

Official references

Git command reference: git-scm.com/docs
Pull and merge behavior: git-scm.com/docs/git-pull
Restore behavior: git-scm.com/docs/git-restore
Pull requests: docs.github.com/en/pull-requests/reference/pull-requests